

GitHub Copilot Cheat Sheet for Eclipse, IntelliJ IDEA, and VS Code

Table of Contents

- [Common GitHub Copilot Shortcuts](#)
- [IDE-specific Shortcuts](#)
 - [Eclipse](#)
 - [IntelliJ IDEA](#)
 - [Visual Studio Code](#)
- [Best Practices and Productivity Tips](#)
- [Language-Specific Examples and Usage Tips](#)
 - [Java](#)
 - [Python](#)
 - [JavaScript](#)

1. Common GitHub Copilot Shortcuts

Action	Windows/Linux	macOS
Accept suggestion	Tab	Tab
Dismiss suggestion	Esc	Esc
Show next suggestion	Alt +]	Option +]
Show previous suggestion	Alt + [	Option + [
Show multiple suggestions	Ctrl + Enter	⌘ + ⇧ + A

Accept next word

Ctrl + →

⌘ + →

- In any IDE, press **Tab** to accept a Copilot suggestion and **Esc** to reject it.
- Use **Alt+] / Alt+[** (Windows/Linux) or **Option+] / Option+[** (macOS) to cycle through alternative inline suggestions.
- To view multiple suggestions at once, press **Ctrl+Enter** (Win/Linux) or **⌘+Shift+A** (macOS).
- You can accept suggestions incrementally: **Ctrl+→** (Win/Linux) or **⌘+→** (macOS) accepts the next word.

2. IDE-specific Shortcuts

Eclipse

After installing the GitHub Copilot plugin for Eclipse (via the Eclipse Marketplace), a Copilot icon appears in the IDE's status bar (bottom-right). Use the following default shortcuts in Eclipse:

Action	Windows/Linux	macOS
Accept suggestion	Tab	Tab
Dismiss suggestion	Esc	Esc
Trigger suggestion	Ctrl+Alt+ /	⌘+⌘+ /
Accept next word	Ctrl+Right	⌘+Right

For example, press **Tab** to accept a Copilot completion or **Esc** to discard it. You can also explicitly invoke Copilot at the cursor: press **Ctrl+Alt+ /** (Windows/Linux) or **Option+Command+ /** (macOS). The “Accept next word” shortcuts (**Ctrl+Right/Cmd+Right**) let you accept the suggestion one word at a time. All keybindings can be customized in **Window > Preferences > General > Keys**.

IntelliJ IDEA

IntelliJ IDEA (and other JetBrains IDEs) uses similar shortcuts. The default Copilot keybindings are shown below:

Action	Windows/Linux	macOS
Accept suggestion	Tab	Tab
Dismiss suggestion	Esc	Esc
Show next suggestion	Alt +]	Option +]
Show previous suggestion	Alt + [	Option + [
Trigger suggestion	Alt+\	⌘ + \
Open Copilot window	Alt+Enter	⌘ + Return

Press **Tab** to accept or **Esc** to dismiss a suggestion. The shortcut **Alt+** (Windows/Linux) or **Option+** (macOS) opens Copilot’s suggestions pane, and **Alt+Enter/ Option+Return** toggles the Copilot Chat or autocompletion window. To change any of these, go to **Settings > Keymap** and search for “Copilot.”

Visual Studio Code

In VS Code with the GitHub Copilot extension, common shortcuts are:

Action	Windows/Linux	macOS
Open Copilot Chat view	Ctrl+Alt+I	⌘+⇧+I
Start new chat session	Ctrl+N	⌘+N
Inline chat in editor/term	Ctrl+I	⌘+I
Accept suggestion	Tab	Tab
Dismiss suggestion	Esc	Esc
Quick Chat (dropdown)	Ctrl+Shift+Alt+L	⇧+⌘+⇧+L

Use **Tab** to accept a completion or **Esc** to reject it. Open the full Chat view with **Ctrl+Alt+I** (Win/Linux) or **⌘+⇧+I** (mac), or the Quick Chat drop-down with **Ctrl+Shift+Alt+L** (Win/Linux) or **⇧+⌘+⇧+L** (mac). You can customize VS Code shortcuts via **File > Preferences > Keyboard Shortcuts** if needed.

3. Best Practices and Productivity Tips

- **Provide rich context:** Keep relevant files open so Copilot has more code context to use. Write clear, high-level comments or docstrings describing what you want (e.g. function purpose or algorithm steps). Use meaningful variable and function names (avoid placeholders like `foo`) to improve suggestion relevance.
- **Explicitly include imports:** Add necessary `import` or `using` statements yourself to steer Copilot toward the correct libraries or framework versions. This avoids Copilot defaulting to outdated APIs.
- **Validate all suggestions:** Copilot is powerful but not infallible. Always review, test, and refactor its output. Remember, “GitHub Copilot is not a compiler” – verify that suggested code compiles and behaves as intended. Treat Copilot as an assistant: accept suggestions first, then inspect and adjust them as needed.
- **Use inline and chat features:** Take advantage of inline chat (e.g. press `Ctrl+I` in VS Code/IDEA) to ask quick questions without leaving your code. For more involved tasks, use **Copilot Chat** in the IDE. Within the chat interface:
 - **Remove irrelevant history:** Delete old or off-topic questions (via the “x” on a message) to keep the conversation focused.
 - **Navigate conversation:** Use the up/down arrow keys to scroll through past prompts and replies, saving time versus scrolling with the mouse.
 - **Use agents:** In VS Code or Visual Studio, mention `@workspace` in your chat prompt to give Copilot full access to your project context.
 - **Highlight or attach code:** Before asking a question, highlight the relevant code in your editor or attach a file (`#file`) so Copilot knows exactly what context to use.
 - **Organize threads:** Keep different topics separate by opening new chat threads (using the “+” button). This prevents cross-talk between unrelated tasks.
- **Use slash commands:** Copilot Chat supports many handy slash commands for common tasks (no need to write the full prompt). For example: `/explain` to explain code, `/fix` to suggest fixes, `/tests` to generate unit tests, `/doc` to create documentation, and `/new` to scaffold new files. These quick commands can save keystrokes and jumpstart many development tasks.
- **Leverage Copilot for debugging:** Before searching online, try describing your bug or error to Copilot Chat. Copilot often quickly pinpoints simple mistakes or misunderstandings. (For example, one user found Copilot Chat identified a forgotten

data variable when debugging a decision tree.)

- **Stay up to date and customize:** Keep your Copilot extension updated (stable or nightly channel) and adjust its settings (such as enabling/disabling for certain languages) to fit your workflow. You can disable Copilot in specific files or languages if needed. Over time, adapt prompts and style to what works best for you – Copilot improves as you iterate on how you ask and code.

4. Language-Specific Examples and Usage Tips

Java

In Java, Copilot works well with class and method stubs and in-code comments. For example, if you type a class declaration like `class Test { ... }`, Copilot will often auto-fill the class body (fields, constructors, methods). Similarly, writing a comment above a method skeleton will prompt Copilot to generate the method's code. In one example, the comment block:

```
// find all images without alternate text
// and give them a red border
void process() {
    // ...
}
```

prompted Copilot to complete the `process()` method implementation based on that description. In practice, provide a complete signature or class context to get the best results. Explicit Java types and imports help Copilot avoid ambiguity. Always review generated Java code for style and correctness, and consider invoking Copilot to write related unit tests once the logic is in place.

Python

GitHub Copilot is especially strong in Python, thanks to Python's concise syntax. A simple comment can trigger a full Python statement. For example, entering:

```
# Print "Hello, World!"
```

in a Python file will lead Copilot to suggest `print("Hello, World!")` when you accept the completion. You can make the task more complex by using multiple comments. For instance, a block of comments describing a function's behavior (filtering offensive words from input text, as shown below) resulted in Copilot generating the entire function body exactly matching the spec:

```
# Ask the user to provide a line of text.
# Scan the text for the following mildly offensive words: arse, bloody, damn, dummy.
# If found, replace all but the first letter of each offensive word with asterisks.
# Print the resulting text.
def main():
    # Copilot completes the function here...
```

This produced a correct implementation with loops and string operations. In short, **describe the task in comments or docstrings**, then use Tab to accept Copilot's suggestion. Copilot respects Python conventions (PEP 8 style is usually applied) and will generate `def` bodies, list comprehensions, exceptions handling, etc., based on the prompt.

JavaScript

In JavaScript, Copilot can fill in both function signatures and logic from comments. For example, typing a function header like:

```
function calculateDaysBetweenDates(begin, end) {
  |
}
```

(with the cursor at `|`) prompts Copilot to complete the function body, calculating days between dates. Similarly, writing task-oriented comments yields code. In a JS file, the user commented:

```
// write a function to
// find all images without alternate text
// and give them a red border
```

and Copilot generated the corresponding DOM-manipulation code to achieve that, which could be accepted with Tab. These features work with modern JS/TypeScript syntax: Copilot can suggest `async/await` patterns, arrow functions, or framework-specific code (e.g. React components) if it sees related imports. When using Copilot in JS, make sure your project's `package.json` or imports are set up so that Copilot knows which libraries (e.g. React, Lodash) you intend to use. As always, review and test the generated code in the browser or Node.js REPL.